1). Aim

To write a Python program to identify eligible applicants based on marks, attendance, and income, and rank them according to aggregate marks and attendance.

2. Problem Description

An applicant is considered eligible if they satisfy all the following conditions:

- Aggregate marks must be greater than or equal to 80.
- Minimum marks in each subject must be greater than or equal to 60.
- Attendance must be greater than or equal to 75%.
- Family income must be less than or equal to ₹2,50,000.

Eligible applicants are ranked in descending order based on aggregate marks. If two applicants have the same aggregate, the applicant with higher attendance gets the higher rank.

3. Python Code

```
applicants = [    {"name": "Asha", "marks": [90, 85, 88],     "attendance": 92, "income": 180000},    {"name": "Ravi", "marks": [82, 84, 80], "attendance": 88, "income": 220000},    {"name": "Mina", "marks": [95, 58, 90],     "attendance": 96, "income": 150000},    {"name": "Kiran", "marks": [85, 85, 85],     "attendance": 90, "income": 200000}]eligible = []for a in applicants:    total = sum(a["marks"])    aggregate = total / len(a["marks"])    ok = (        aggregate >= 80        and min(a["marks"]) >= 60        and a["attendance"] >= 75        and a["income"] <= 250000    )    if ok:        eligible.append(            (aggregate, a["attendance"], a["name"])        )eligible.sort(    key=lambda x: (x[0], x[1]), reverse=True)if not eligible:    print("No eligible applicants")else:    for rank, (marks, attendance, name) in enumerate(eligible, 1):        print(            rank, name,            "Aggregate:", round(marks, 2),            "Attendance:", attendance        )
```

4. Algorithm

1. Start.
2. Store applicant details such as name, marks, attendance, and income.
3. Create an empty list called `eligible`.
4. For each applicant:
    - Calculate the total marks.
    - Calculate the aggregate marks.
    - Check all eligibility conditions.
5. If all conditions are satisfied, add the applicant to the eligible list.
6. Sort eligible applicants in descending order based on aggregate marks and attendance.
7. If the eligible list is empty, print "No eligible applicants".
8. Otherwise, display the rank, name, aggregate marks, and attendance of each eligible applicant.
9. Stop.

```
BEGIN

  CREATE applicants list
  CREATE empty eligible list

  FOR each applicant in applicants

    total = SUM of applicant marks

    aggregate = total / number of subjects

    IF aggregate >= 80 AND
      minimum marks >= 60 AND
      attendance >= 75 AND
      income <= 250000 THEN

       ADD applicant details to eligible list

    END IF

  END FOR

  SORT eligible list in descending order
  based on aggregate and attendance

  IF eligible list is empty THEN

    PRINT "No eligible applicants"

  ELSE
```

FOR each applicant in eligible list

PRINT rank, name, aggregate,

and attendance

END FOR

END IF

END6. Input

The program uses the following applicant details as input:

Name	Marks	Attendance	Income
Asha	90, 85, 88	92	180000
Ravi	82, 84, 80	88	220000
Mina	95, 58, 90	96	150000
Kiran	85, 85, 85	90	200000

7. Output

```
1 Asha Aggregate: 87.67 Attendance: 92
2 Kiran Aggregate: 85.0 Attendance: 90
3 Ravi Aggregate: 82.0 Attendance: 88
```

8. Flowchart

START

Input applicant details

Calculate total and aggregate marks

Check eligibility conditions

Aggregate ≥ 80, minimum mark ≥ 60, attendance ≥ 75, income ≤ 250000

YES

Add applicant to eligible list

NO

Reject applicant

Sort eligible applicants by aggregate and attendance

Display rank, name, aggregate, and attendance

STOP

9. Output Screenshot

Build a Grade Calculator with Functions - by Ardit Sulce

dailypythonprojects.substack.com

Here is the actual screenshot generated from the program's output:

Download Output Screenshot

10. Conclusion

Thus, the Python program successfully checks the eligibility of applicants based on their aggregate marks, minimum subject marks, attendance, and family income.

It identifies eligible applicants and ranks them in descending order based on aggregate marks and attendance. The program demonstrates the use of lists, dictionaries, loops, conditional statements, sorting, and functions in Python.

2).1 Aim

To write a Python program to calculate the gross salary, salary deductions, and net salary of an employee using basic salary, overtime hours, and incentive.

2. Python Code

```
basic = float(input("Basic salary: "))hours = float(input("Overtime hours:
"))incentive = float(input("Incentive: "))if basic < 0 or hours < 0 or
incentive < 0:    print("Inputs cannot be negative")else:    hra = basic *
0.20    da = basic * 0.10    overtime = hours * 200    gross = basic + hra
+ da + overtime + incentive    deduction = 0    deduction += max(0,
min(gross, 40000) - 20000) * 0.05    deduction += max(0, gross - 40000) *
0.10    net = gross - deduction    print("\n--- SALARY STATEMENT ---")
print("Basic: ", round(basic, 2))    print("HRA: ", round(hra, 2), " DA: ",
round(da, 2))    print("Overtime: ", round(overtime, 2))
print("Incentive: ", round(incentive, 2))    print("Gross salary: ",
round(gross, 2))    print("Deduction: ", round(deduction, 2))    print("Net
salary: ", round(net, 2))
```

3. Algorithm

1. Start.
2. Read basic salary, overtime hours, and incentive.
3. Check whether any input is negative.
4. If any input is negative, display "Inputs cannot be negative".
5. Otherwise:
    - Calculate HRA as 20% of basic salary.

    1.
    - Calculate DA as 10% of basic salary.
    - Calculate overtime pay as overtime hours × 200.
    - Calculate gross salary by adding basic salary, HRA, DA, overtime pay, and incentive.
2. Calculate salary deduction:
    - 5% deduction on the portion of gross salary between ₹20,000 and ₹40,000.
    - 10% deduction on the portion above ₹40,000.
3. Calculate net salary by subtracting deduction from gross salary.
4. Display the salary statement.
5. Stop.

4. Pseudocode

```
BEGIN

    INPUT basic salary
    INPUT overtime hours
    INPUT incentive

    IF basic < 0 OR hours < 0 OR incentive < 0 THEN

        PRINT "Inputs cannot be negative"

    ELSE

        hra = basic * 0.20
        da = basic * 0.10
        overtime = hours * 200

        gross = basic + hra + da + overtime + incentive

        deduction = 0

        deduction = deduction +
                    5% of salary between 20000 and 40000

        deduction = deduction +
                    10% of salary above 40000

        net = gross - deduction

        PRINT basic salary
        PRINT HRA
        PRINT DA
        PRINT overtime
        PRINT incentive
        PRINT gross salary
        PRINT deduction
```

```
        PRINT net salary

    END IF

END
```

5. Input

The program takes the following input values from the user.

Input	Value
Basic salary	30000
Overtime hours	5
Incentive	2000

6. Calculation

- HRA = 30000 × 20% = ₹6,000
- DA = 30000 × 10% = ₹3,000
- Overtime = 5 × 200 = ₹1,000
- Gross salary = 30000 + 6000 + 3000 + 1000 + 2000 = ₹42,000

Deduction:

- 5% of ₹20,000 = ₹1,000
- 10% of ₹2,000 = ₹200
- Total deduction = ₹1,200

Net salary:

- Net salary = Gross salary − Deduction
- Net salary = 42000 − 1200 = ₹40,800

7. Output

```
Basic salary: 30000
Overtime hours: 5
Incentive: 2000

--- SALARY STATEMENT ---
Basic:  30000.0
HRA:  6000.0  DA:  3000.0
Overtime:  1000.0
Incentive:  2000.0
Gross salary:  42000.0
Deduction:  1200.0
Net salary:  40800.0
```

8. Flowchart

START

Input basic salary, overtime hours, and incentive

Check if any input is negative

NO

Calculate HRA, DA, and overtime
Calculate gross salary
Calculate deduction and net salary
Display salary statement

YES

Print "Inputs cannot be negative"

STOP

9. Output Screenshot

Primeiro Menu com Python Calculo Salario Bruto e Liquido | Valdemar Taborda | Python | DIO
dio.me

The screenshot below represents the output generated using the sample input values.

Download Output Screenshot

10. Conclusion

Thus, the Python program successfully calculates an employee's gross salary, salary deductions, and net salary based on basic salary, overtime hours, and incentive.

The program also checks for negative inputs and displays an appropriate error message. It demonstrates the use of input statements, arithmetic operations, conditional statements, and built-in functions in Python.

salary_calculation_output_screenshot.pngImage

1.

- Calculate DA as 10% of basic salary.
- Calculate overtime pay as overtime hours × 200.
- Calculate gross salary by adding basic salary, HRA, DA, overtime pay, and incentive.

3.1Aim

To write a Python program to select the most economical mobile data plan that satisfies the user's required validity, expected data usage, and budget.

2. Python Code

```
plans = [    {"name": "Plan A", "price": 199, "daily": 1.5, "days": 28},
{"name": "Plan B", "price": 299, "daily": 2.0, "days": 28},    {"name":
"Plan C", "price": 399, "daily": 2.5, "days": 56},] AimTo write a Python
program to select the most economical mobile data plan that satisfies the
user's required validity, expected data usage, and budget.2. Python
Coderequired_days = int(input("Required validity in days: "))usage =
float(input("Expected total data usage (GB): "))budget =
float(input("Budget: "))eligible = []if required_days <= 0 or usage < 0 or
budget < 0:    print("Enter valid values")else:    for p in plans:
included = p["daily"] * p["days"]        excess = max(0, usage - included)
cost = p["price"] + excess * 20        if p["days"] >= required_days and
cost <= budget:            eligible.append((cost, p["name"], excess))    if
eligible:        eligible.sort()        cost, name, excess = eligible[0]
print("Most economical eligible plan:", name)        print("Estimated cost:
", round(cost, 2))        print("Excess data: ", round(excess, 2), "GB")
else:        print("No plan satisfies validity and budget constraints")
```

3. Algorithm

1. Start.
2. Store the available mobile data plans with their price, daily data, and validity.
3. Read the required validity, expected total data usage, and budget.
4. Check whether the input values are valid.
5. If the values are invalid, display "Enter valid values".
6. Otherwise, for each plan:
    - Calculate the total included data.
    - Calculate excess data usage.
    - Calculate the estimated cost.
    - Check whether the plan satisfies the validity and budget conditions.
7. Add eligible plans to the eligible list.
8. Sort eligible plans in ascending order based on cost.
9. If eligible plans are available, display the most economical plan, estimated cost, and excess data.
10. Otherwise, display "No plan satisfies validity and budget constraints".
11. Stop.

4. Pseudocode

```
BEGIN

    STORE available plans

    INPUT required validity
    INPUT expected total data usage
    INPUT budget
```

```
    CREATE empty eligible list

    IF required validity <= 0 OR
       usage < 0 OR budget < 0 THEN

        PRINT "Enter valid values"

    ELSE

        FOR each plan in plans

            included = daily data * validity

            excess = MAX(0, usage - included)

            cost = price + excess * 20

            IF plan validity >= required validity AND
               cost <= budget THEN

                ADD cost, plan name, excess
                TO eligible list

            END IF

        END FOR

        IF eligible list is not empty THEN

            SORT eligible list by cost in ascending order

            SELECT first plan

            PRINT plan name
            PRINT estimated cost
            PRINT excess data

        ELSE

            PRINT "No plan satisfies validity and budget constraints"

        END IF

    END IF

END
```

5. Input

The program takes the following input values from the user.

Input	Value
Required validity	28 days
Expected total data usage	50 GB
Budget	₹500

6. Calculation

Available plans:

Plan	Price	Daily Data	Validity
Plan A	₹199	1.5 GB	28 days
Plan B	₹299	2 GB	28 days
Plan C	₹399	2.5 GB	56 days

Plan A

- Included data = 1.5 × 28 = 42 GB
- Excess data = 50 − 42 = 8 GB
- Estimated cost = 199 + (8 × 20) = ₹359

Plan B

- Included data = 2 × 28 = 56 GB
- Excess data = 0 GB
- Estimated cost = ₹299

Plan C

- Included data = 2.5 × 56 = 140 GB
- Excess data = 0 GB
- Estimated cost = ₹399

All three plans satisfy the required validity of 28 days and budget of ₹500.

Plan B is selected because it has the lowest estimated cost of ₹299.

7. Output

```
Required validity in days: 28
Expected total data usage (GB): 50
Budget: 500

Most economical eligible plan: Plan B
Estimated cost:  299
Excess data:  0 GB
```

8. Flowchart

START

Store mobile data plans

Input required validity, usage, and budget

Check input validity

VALID

Calculate included data, excess data, and cost

Check validity and budget

Add eligible plans to list

Sort by cost

Display most economical plan

INVALID

Print "Enter valid values"

STOP

9. Output Screenshot

The output screenshot file could not be created right now because data analysis is temporarily unavailable. It is expected to be available again in about 22 hours and 31 minutes, though this is only an estimate.

For now, you can use the output text in Section 7 as the reference for your practical record.

10. Conclusion

Thus, the Python program successfully selects the most economical mobile data plan based on the user's required validity, expected data usage, and budget.

It calculates the included data, excess data, and estimated cost for each plan. It then selects the lowest-cost plan that satisfies the given conditions.

The program demonstrates the use of lists, dictionaries, loops, conditional statements, sorting, and built-in functions in Python.

4)1. Aim

To write a Python program to calculate railway reservation fares based on passenger age, distance, and travel class, and calculate the cancellation penalty and estimated refund.

2. Python Code

```
distance = float(input("Distance in km: "))travel_class = input("Class
(Sleeper/AC): ").strip().lower()count = int(input("Number of passengers:
```

```
"))if distance <= 0 or count <= 0 or travel_class not in ("sleeper", "ac"):
print("Invalid travel details")else:    rate = 1 if travel_class ==
"sleeper" else 2    total_fare = 0    total_penalty = 0    cancelled =
input("Cancel reservation? (yes/no): ").lower() == "yes"    for i in
range(count):        age = int(input(f"Age of passenger {i + 1}: "))
if age < 0:           print("Invalid age")           continue        if
age < 5:           fare = 0        else:           base = distance * rate
if age >= 60:                base *= 0.80           fare = base + 40
penalty = fare * 0.25 if cancelled else 0        total_fare += fare
total_penalty += penalty
```

3. Algorithm

1. Start.
2. Read the travel distance, travel class, and number of passengers.
3. Check whether the travel details are valid.
4. If the details are invalid, display "Invalid travel details".
5. Otherwise:
    - Set the fare rate according to the travel class.
    - Initialize total fare and total penalty to zero.
    - Ask whether the reservation is cancelled.
6. For each passenger:
    - Read the passenger's age.
    - If age is negative, display "Invalid age".
    - If age is below 5, set fare to zero.
    - Otherwise, calculate the base fare using distance and class rate.
    - If age is 60 or above, apply a 20% discount to the base fare.
    - Add ₹40 reservation charge to the fare.
    - If reservation is cancelled, calculate 25% cancellation penalty.
    - Add the passenger fare and penalty to the totals.
    - Display passenger fare and cancellation penalty.
7. Display total fare, total cancellation penalty, and estimated refund.
8. Stop.

4. Pseudocode

```
BEGIN

    INPUT distance
    INPUT travel_class
    INPUT number of passengers

    IF distance <= 0 OR count <= 0 OR
       travel_class is invalid THEN

        PRINT "Invalid travel details"

    ELSE

        IF travel_class = "sleeper" THEN
            rate = 1
        ELSE
            rate = 2
        END IF
```

```
        total_fare = 0
        total_penalty = 0

        INPUT cancellation status

        FOR each passenger

            INPUT age

            IF age < 0 THEN
                PRINT "Invalid age"
                CONTINUE
            END IF

            IF age < 5 THEN
                fare = 0
            ELSE
                base = distance * rate

                IF age >= 60 THEN
                    base = base * 0.80
                END IF

                fare = base + 40
            END IF

            IF reservation is cancelled THEN
                penalty = fare * 0.25
            ELSE
                penalty = 0
            END IF

            total_fare = total_fare + fare
            total_penalty = total_penalty + penalty

            PRINT passenger fare
            PRINT cancellation penalty

        END FOR

        refund = total_fare - total_penalty

        PRINT total_fare
        PRINT total_penalty
        PRINT refund

    END IF

END
```

5. Input

The program takes the following input values from the user.

Input	Value
Distance in km	500
Class	AC

Number of passengers	3
Cancel reservation?	yes
Age of passenger 1	30
Age of passenger 2	65
Age of passenger 3	4

6. Calculation

Passenger 1 (Age 30)

- Base fare = 500 × 2 = ₹1,000
- Reservation charge = ₹40
- Fare = ₹1,040
- Cancellation penalty = 1040 × 25% = ₹260

Passenger 2 (Age 65)

- Base fare = 500 × 2 = ₹1,000
- Senior citizen discount = 20%
- Discounted base fare = 1000 × 0.80 = ₹800
- Reservation charge = ₹40
- Fare = ₹840
- Cancellation penalty = 840 × 25% = ₹210

Passenger 3 (Age 4)

- Age is below 5.
- Fare = ₹0
- Cancellation penalty = ₹0

Total

- Total fare = 1040 + 840 + 0 = ₹1,880
- Total cancellation penalty = 260 + 210 + 0 = ₹470
- Estimated refund = 1880 − 470 = ₹1,410

7. Output

```
Distance in km: 500
Class (Sleeper/AC): AC
Number of passengers: 3
Cancel reservation? (yes/no): yes
Age of passenger 1: 30
Passenger 1 fare:  1040.0 cancellation penalty:  260.0
Age of passenger 2: 65
Passenger 2 fare:  840.0 cancellation penalty:  210.0
Age of passenger 3: 4
Passenger 3 fare:  0 cancellation penalty:  0.0
Total fare:  1880.0
```

```
Total cancellation penalty:  470.0
Estimated refund after penalty:  1410.0
```

8. Flowchart

START

Input distance, class, and passenger count

Check travel details

VALID

Set class rate and initialize totals

Input cancellation status

Input passenger age

Calculate fare based on age and class

Calculate cancellation penalty

Update total fare and penalty

INVALID

Print "Invalid travel details"

Display total fare, total penalty, and estimated refund

STOP

9. Output Screenshot

The screenshot file could not be generated at this time because data analysis is temporarily unavailable. It is expected to become available again in about 22 hours and 31 minutes, though this is only an estimate.

For now, the output in Section 7 shows the exact expected terminal output for the sample input values.

10. Conclusion

Thus, the Python program successfully calculates railway reservation fares for passengers based on distance, travel class, and age.

It applies the appropriate fare rate, senior citizen discount, and reservation charge. It also calculates the cancellation penalty and estimated refund when a reservation is cancelled.

The program demonstrates the use of input statements, conditional statements, loops, arithmetic operations, and built-in functions in Python.

5).1 Aim

To write a Python program to calculate manufacturing production costs, revenue, profit or loss, and profit percentage, and check whether the target profit percentage is achieved.

2. Python Code

```
units = int(input("Units produced: "))material_unit = float(input("Material
cost per unit: "))wastage_rate = float(input("Wastage percentage: "))labour
= float(input("Total labour cost: "))machine = float(input("Machine cost:
"))selling_price = float(input("Selling price per unit: "))target =
float(input("Target profit percentage on cost: "))values = [
material_unit, wastage_rate, labour,    machine, selling_price, target]if
units <= 0 or any(v < 0 for v in values):    print("Enter valid non-
negative values and positive units")else:    material = units *
material_unit    wastage = material * wastage_rate / 100    total_cost =
material + wastage + labour + machine    revenue = units * selling_price
profit = revenue - total_cost    margin = profit / total_cost * 100 if
total_cost else 0    print("Material cost: ", round(material, 2))
print("Wastage cost: ", round(wastage, 2))    print("Total production cost:
", round(total_cost, 2))    print("Revenue: ", round(revenue, 2))
print("Profit/Loss: ", round(profit, 2))    print("Profit on cost: ",
round(margin, 2), "%")    print(        "Target achieved"
```

3. Algorithm

1. Start.
2. Read the number of units produced.
3. Read material cost per unit, wastage percentage, labour cost, machine cost, selling price per unit, and target profit percentage.
4. Check whether the units are positive and all other input values are non-negative.
5. If inputs are invalid, display an error message.
6. Otherwise:
    - Calculate material cost.
    - Calculate wastage cost.
    - Calculate total production cost.
    - Calculate revenue.
    - Calculate profit or loss.
    - Calculate profit percentage on cost.
7. Compare the calculated profit percentage with the target profit percentage.
8. Display all cost and profit details.
9. Display whether the target is achieved.
10. Stop.

4. Pseudocode

```
BEGIN
```

```
    INPUT units
    INPUT material cost per unit
    INPUT wastage percentage
    INPUT labour cost
    INPUT machine cost
    INPUT selling price per unit
    INPUT target profit percentage

    IF units <= 0 OR any input value < 0 THEN

        PRINT "Enter valid non-negative values and positive units"

    ELSE

        material = units * material_unit

        wastage = material * wastage_rate / 100

        total_cost = material + wastage + labour + machine

        revenue = units * selling_price

        profit = revenue - total_cost

        IF total_cost is not zero THEN
            margin = profit / total_cost * 100
        ELSE
            margin = 0
        END IF

        PRINT material cost
        PRINT wastage cost
        PRINT total production cost
        PRINT revenue
        PRINT profit or loss
        PRINT profit percentage

        IF margin >= target THEN
            PRINT "Target achieved"
        ELSE
            PRINT "Target not achieved"
        END IF

    END IF

END
```

5. Input

The program takes the following input values from the user.

Input	Value
Units produced	100
Material cost per unit	₹50
Wastage percentage	10%
Total labour cost	₹2,000

Machine cost	₹1,000
Selling price per unit	₹100
Target profit percentage	30%

6. Calculation

Material cost

Material cost = Units produced × Material cost per unit

= 100 × 50

= ₹5,000

Wastage cost

Wastage cost = Material cost × Wastage percentage / 100

= 5000 × 10 / 100

= ₹500

Total production cost

Total production cost = Material cost + Wastage cost + Labour cost + Machine cost

= 5000 + 500 + 2000 + 1000

= ₹8,500

Revenue

Revenue = Units produced × Selling price per unit

= 100 × 100

= ₹10,000

Profit

Profit = Revenue − Total production cost

= 10000 − 8500

= ₹1,500

Profit percentage

Profit percentage = Profit / Total production cost × 100

= 1500 / 8500 × 100

= 17.65%

The target profit percentage is 30%.

Since 17.65% is less than 30%, the target is not achieved.

7. Output

```
Units produced: 100
Material cost per unit: 50
Wastage percentage: 10
Total labour cost: 2000
Machine cost: 1000
Selling price per unit: 100
Target profit percentage on cost: 30

Material cost:  5000.0
Wastage cost:  500.0
Total production cost:  8500.0
Revenue:  10000.0
Profit/Loss:  1500.0
Profit on cost:  17.65 %
Target not achieved
```

8. Flowchart

START

Input units, material cost, wastage, labour, machine cost, selling price, and target profit

Check input validity

VALID

Calculate material and wastage costs

Calculate total production cost and revenue

Calculate profit or loss and profit percentage

Compare profit percentage with target

Display results and target status

INVALID

Print "Enter valid non-negative values and positive units"

STOP

9. Output Screenshot

The output screenshot file could not be created because data analysis is temporarily unavailable. It is expected to be available again in about 22 hours and 24 minutes, though this is only an estimate.

The output text in Section 7 provides the expected program output for the sample input values.

10. Conclusion

Thus, the Python program successfully calculates the manufacturing production cost, revenue, profit or loss, and profit percentage.

It also compares the calculated profit percentage with the target profit percentage and displays whether the target is achieved.

The program demonstrates the use of input statements, arithmetic operations, conditional statements, lists, and built-in functions in Python

6).1Aim

To write a Python program to convert travel expenses from foreign currency into home currency, calculate the total spending including conversion fees, identify the largest expense category, and check whether the total spending is within the travel budget.

2. Python Code

```
budget = float(input("Travel budget in home currency: "))fee_rate =
float(input("Conversion fee percentage: "))expenses = []n =
int(input("Number of expense categories: "))if budget < 0 or fee_rate < 0
or n <= 0:    print("Enter valid values")else:    total = 0    for i in
range(n):        category = input("Expense category: ")
foreign_amount = float(input("Amount in foreign currency: "))        rate =
float(input("Exchange rate (home currency per 1 foreign): "))        if
foreign_amount < 0 or rate <= 0:            print("Invalid amount or
exchange rate; skipped")            continue        converted =
foreign_amount * rate        final = converted * (1 + fee_rate / 100)
expenses.append((category, final))        total += final    if expenses:
largest = max(expenses, key=lambda x: x[1])        print("Total spending:
", round(total, 2))        print("Largest expense category:", largest[0],
"(amount:", round(largest[1], 2), ")")        if total <= budget:
print("Budget remaining:", round(budget - total, 2))        else:
print("Over budget by:", round(total - budget, 2))    else:
print("No valid expenses entered")
```

3. Algorithm

1. Start.
2. Read the travel budget.
3. Read the conversion fee percentage.
4. Read the number of expense categories.
5. Check whether the entered values are valid.
6. Initialize total spending to zero.
7. Repeat for each expense category:
    - Read the category name.
    - Read the foreign currency amount.
    - Read the exchange rate.
    - Check whether the amount and rate are valid.
    - Convert the amount into home currency.
    - Add the conversion fee.
    - Store the category and final expense.
    - Add the expense to total spending.
8. Find the largest expense category.
9. Display the total spending and largest expense.
10. Compare total spending with the budget.
11. Display the remaining budget or amount over budget.
12. Stop.

4. Pseudocode

```
START

INPUT budget
INPUT fee_rate
INPUT n

IF budget < 0 OR fee_rate < 0 OR n <= 0 THEN
    PRINT "Enter valid values"
ELSE
    total = 0
    expenses = empty list

    FOR each expense category from 1 to n
        INPUT category
        INPUT foreign_amount
        INPUT rate

        IF foreign_amount < 0 OR rate <= 0 THEN
            PRINT "Invalid amount or exchange rate; skipped"
            CONTINUE
        END IF

        converted = foreign_amount * rate
        final = converted * (1 + fee_rate / 100)

        ADD (category, final) TO expenses
        total = total + final
    END FOR
```

```
    IF expenses is not empty THEN
        largest = expense with maximum amount

        PRINT total
        PRINT largest expense category and amount

        IF total <= budget THEN
            PRINT budget - total
        ELSE
            PRINT total - budget
        END IF
    ELSE
        PRINT "No valid expenses entered"
    END IF
END IF

STOP
```

5. Input

Sample input values:

```
Travel budget in home currency: 50000
Conversion fee percentage: 2
Number of expense categories: 3

Expense category: Hotel
Amount in foreign currency: 200
Exchange rate (home currency per 1 foreign): 80

Expense category: Food
Amount in foreign currency: 100
Exchange rate (home currency per 1 foreign): 80

Expense category: Shopping
Amount in foreign currency: 150
Exchange rate (home currency per 1 foreign): 80
```

6. Calculation

Conversion formula:

$$\text{Converted Amount}=\text{Foreign Amount}\times\text{Exchange Rate}$$

Final expense including conversion fee:

$$\text{Final Amount}=\text{Converted Amount}\times(1+\text{Fee Rate}/100)$$

Category	Converted amount	Fee (2%)	Final amount
Hotel	16,000	320	16,320
Food	8,000	160	8,160
Shopping	12,000	240	12,240
Total	36,000	720	36,720

Budget = 50,000

Remaining budget = 50,000 − 36,720 = 13,280

Largest expense category = Hotel

7. Output

```
Travel budget in home currency: 50000
Conversion fee percentage: 2
Number of expense categories: 3
Expense category: Hotel
Amount in foreign currency: 200
Exchange rate (home currency per 1 foreign): 80
Expense category: Food
Amount in foreign currency: 100
Exchange rate (home currency per 1 foreign): 80
Expense category: Shopping
Amount in foreign currency: 150
Exchange rate (home currency per 1 foreign): 80

Total spending:  36720.0
Largest expense category: Hotel (amount: 16320.0 )
Budget remaining: 13280.0
```

8. Flowchart

START

Input budget, fee rate and number of expenses

Are the input values valid?

Yes

Input category, amount and exchange rate
Convert currency and add fee
Store expense and update total
Repeat for all categories
Find largest expense and display total

Total ≤ Budget?

Display remaining budget or over-budget amount

No

Display "Enter valid values"

STOP

9. Output Screenshot

I can’t generate a screenshot file in this response. You can create it easily by running the program in Python IDLE or VS Code and capturing the output.

To take a screenshot on Windows:

1. Run the Python program.
2. Enter the sample input values above.
3. When the output appears, press `Win + Shift + S`.
4. Select the output area and save the screenshot.

10. Conclusion

Thus, the Python program successfully converts foreign currency expenses into home currency, includes conversion fees, calculates total spending, identifies the largest expense category, and checks whether the travel budget is sufficient. It uses input statements, arithmetic operations, loops, conditional statements, lists, and built-in functions.

7).1Aim

To write a Python program to check whether the available stock is sufficient to meet the required quantity. If sufficient, it displays the remaining stock. Otherwise, it displays the additional quantity needed.

2. Python Code

```
stock = int(input("Enter available stock: "))required = int(input("Enter
required quantity: "))if stock >= required:    print("Stock is sufficient")
remaining = stock - required    print("Remaining stock =", remaining)else:
print("Stock is insufficient")    shortage = required - stock
print("Additional stock needed =", shortage)
```

3. Algorithm

1. Start.
2. Input available stock.
3. Input required quantity.
4. Compare available stock with required quantity.
5. If stock is greater than or equal to required quantity:
    - Display “Stock is sufficient”.
    - Calculate remaining stock.
    - Display remaining stock.
6. Otherwise:
    - Display “Stock is insufficient”.
    - Calculate additional stock needed.
    - Display additional stock needed.
7. Stop.

4. Pseudocode

```
START

INPUT stock
INPUT required

IF stock >= required THEN
    PRINT "Stock is sufficient"
    remaining = stock - required
    PRINT remaining stock
ELSE
    PRINT "Stock is insufficient"
    shortage = required - stock
    PRINT additional stock needed
END IF

STOP
```

5. Input

Sample input:

```
Enter available stock: 100
Enter required quantity: 75
```

6. Calculation

Available stock = 100

Required quantity = 75

Remaining stock = 100 − 75 = 25

7. Output

```
Enter available stock: 100
Enter required quantity: 75
Stock is sufficient
Remaining stock = 25
```

Another example (insufficient stock)

Input:

```
Enter available stock: 50
Enter required quantity: 80
```

Output:

```
Stock is insufficient
Additional stock needed = 30
```

8. Flowchart

START

Input available stock and required quantity

Is stock ≥ required quantity?

YES

Display "Stock is sufficient"

Calculate remaining stock

Display remaining stock

NO

Display "Stock is insufficient"

Calculate shortage

Display additional stock needed

STOP

9. Output Screenshot

I can’t generate a screenshot file in this response. To take one:

1. Run the program in Python IDLE or VS Code.
2. Enter the sample inputs.
3. After the output appears, press `Win + Shift + S`.
4. Select the output area and save the screenshot.

10. Conclusion

Thus, the Python program checks whether the available stock is sufficient to meet the required quantity. It displays the remaining stock when sufficient or the additional stock needed when insufficient. The program uses input statements, arithmetic operations, and conditional statements.

8).1Aim

To write a Python program to calculate the total cost of an event by adding venue, food, transport, and equipment costs, and check whether the event is within the given budget of ₹10,000.

2. Python Code

```
venue = float(input("Enter venue cost: "))food = float(input("Enter food
cost: "))transport = float(input("Enter transport cost: "))equipment =
float(input("Enter equipment cost: "))total = venue + food + transport +
equipmentprint("Total event cost =", total)if total <= 10000:
print("Event is within budget")else:    print("Event budget exceeded")
```

3. Algorithm

1. Start.
2. Input venue cost.
3. Input food cost.
4. Input transport cost.
5. Input equipment cost.
6. Calculate the total event cost by adding all costs.
7. Display the total event cost.
8. Check whether the total cost is less than or equal to ₹10,000.
9. If yes, display “Event is within budget”.
10. Otherwise, display “Event budget exceeded”.
11. Stop.

4. Pseudocode

```
START

INPUT venue
INPUT food
INPUT transport
INPUT equipment

total = venue + food + transport + equipment

PRINT total

IF total <= 10000 THEN
    PRINT "Event is within budget"
ELSE
    PRINT "Event budget exceeded"
END IF

STOP
```

5. Input

Sample input:

```
Enter venue cost: 3000
Enter food cost: 4000
Enter transport cost: 1500
Enter equipment cost: 1000
```

6. Calculation

Total event cost:

\[3000+4000+1500+1000=9500 \]

Budget = ₹10,000

Since ₹9,500 ≤ ₹10,000, the event is within budget.

7. Output

```
Enter venue cost: 3000
Enter food cost: 4000
Enter transport cost: 1500
Enter equipment cost: 1000
Total event cost = 9500.0
Event is within budget
```

8. Flowchart

START

Input venue, food, transport and equipment costs
Calculate total event cost

Is total cost ≤ ₹10,000?

YES

Display "Event is within budget"

NO

Display "Event budget exceeded"

STOP

9. Output Screenshot

I can’t generate a screenshot file in this response. You can take one by following these steps:

1. Run the program in Python IDLE or VS Code.
2. Enter the sample input values.
3. After the output appears, press `Win + Shift + S`.
4. Select the output area and save the screenshot.

10. Conclusion

Thus, the Python program calculates the total event cost by adding venue, food, transport, and equipment expenses. It compares the total cost with the fixed budget of ₹10,000 and displays whether the event is within budget or exceeds the budget. The program uses input statements, arithmetic operations, and conditional statements.

9).1 Aim

To write a Python program to calculate the total hospital bill by adding consultation fees, test charges, and treatment charges. The program also checks whether the total bill is ₹5,000 or more and displays “High bill” or “Normal bill”.

2. Python Code

```
consultation = float(input("Enter consultation fee: "))tests =
float(input("Enter test charges: "))treatment = float(input("Enter
treatment charges: "))total = consultation + tests + treatmentprint("Total
hospital bill =", total)if total >= 5000:    print("High bill")else:
print("Normal bill")
```

3. Algorithm

1. Start.
2. Input consultation fee.
3. Input test charges.
4. Input treatment charges.
5. Calculate the total hospital bill.
6. Display the total bill.
7. Check whether the total bill is greater than or equal to ₹5,000.
8. If yes, display “High bill”.
9. Otherwise, display “Normal bill”.
10. Stop.

4. Pseudocode

```
START

INPUT consultation
INPUT tests
INPUT treatment

total = consultation + tests + treatment

PRINT total

IF total >= 5000 THEN
    PRINT "High bill"
ELSE
    PRINT "Normal bill"
END IF

STOP
```

5. Input

Sample input:

```
Enter consultation fee: 1000
```

```
Enter test charges: 2500
Enter treatment charges: 2000
```

6. Calculation

Total hospital bill:

\[1000+2500+2000=5500 \]

Since ₹5,500 ≥ ₹5,000, the bill is classified as a high bill.

7. Output

```
Enter consultation fee: 1000
Enter test charges: 2500
Enter treatment charges: 2000
Total hospital bill = 5500.0
High bill
```

8. Flowchart

START

Input consultation fee, test charges and treatment charges
Calculate total hospital bill
Display total bill

Is total ≥ ₹5,000?

YES

Display "High bill"

NO

Display "Normal bill"

STOP

9. Output Screenshot

I can’t generate a screenshot file in this response. To take one:

1. Run the Python program in IDLE or VS Code.
2. Enter the sample inputs.
3. After the output appears, press `Win + Shift + S`.
4. Select the output area and save the screenshot.

10. Conclusion

Thus, the Python program calculates the total hospital bill by adding consultation fees, test charges, and treatment charges. It checks whether the bill is ₹5,000 or more and displays “High bill” or “Normal bill”. The program uses input statements, arithmetic operations, and conditional statements.

10).1Aim

To write a Python program to calculate an employee’s gross salary and net salary based on basic salary, HRA, DA, overtime, incentive, and deductions. The program also displays a detailed salary statement.

2. Python Code

```
basic = float(input("Basic salary: "))hours = float(input("Overtime hours: "))incentive = float(input("Incentive: "))if basic < 0 or hours < 0 or incentive < 0:    print("Inputs cannot be negative")else:    hra = basic * 0.20    da = basic * 0.10    overtime = hours * 200    gross = basic + hra + da + overtime + incentive    deduction = 0    deduction += max(0, min(gross, 40000) - 20000) * 0.05    deduction += max(0, gross - 40000) * 0.10    net = gross - deduction    print("\n--- SALARY STATEMENT ---")print("Basic: ", round(basic, 2))    print("HRA: ", round(hra, 2), " DA: ", round(da, 2))    print("Overtime: ", round(overtime, 2))print("Incentive: ", round(incentive, 2))    print("Gross salary: ", round(gross, 2))    print("Deduction: ", round(deduction, 2))    print("Net salary: ", round(net, 2))
```

3. Algorithm

1. Start.
2. Input basic salary.
3. Input overtime hours.
4. Input incentive.
5. Check whether any input value is negative.
6. If negative, display an error message.
7. Otherwise:
    - Calculate HRA as 20% of basic salary.
    - Calculate DA as 10% of basic salary.
    - Calculate overtime pay at ₹200 per hour.
    - Calculate gross salary.
    - Calculate deductions based on gross salary.
    - Calculate net salary.
8. Display the salary statement.
9. Stop.

4. Pseudocode

```
START

INPUT basic
INPUT hours
INPUT incentive
```

```
IF basic < 0 OR hours < 0 OR incentive < 0 THEN
    PRINT "Inputs cannot be negative"
ELSE
    hra = basic * 0.20
    da = basic * 0.10
    overtime = hours * 200

    gross = basic + hra + da + overtime + incentive

    deduction = 0

    deduction = deduction +
                MAX(0, MIN(gross, 40000) - 20000) * 0.05

    deduction = deduction +
                MAX(0, gross - 40000) * 0.10

    net = gross - deduction

    PRINT basic, hra, da, overtime
    PRINT incentive
    PRINT gross
    PRINT deduction
    PRINT net
END IF

STOP
```

5. Input

Sample input:

```
Basic salary: 30000
Overtime hours: 5
Incentive: 2000
```

6. Calculation

Component	Calculation	Amount
Basic salary	Given	₹30,000
HRA	30,000 × 20%	₹6,000
DA	30,000 × 10%	₹3,000
Overtime	5 × 200	₹1,000
Incentive	Given	₹2,000
Gross salary	Sum of all components	₹42,000

Deduction calculation

- 5% deduction on salary between ₹20,000 and ₹40,000 = ₹1,000
- 10% deduction on salary above ₹40,000 = ₹200
- Total deduction = ₹1,200

Net salary:

\[\text{Net Salary} = 42000 - 1200 = ₹40800 \]

7. Output

```
Basic salary: 30000
Overtime hours: 5
Incentive: 2000

--- SALARY STATEMENT ---
Basic:  30000.0
HRA:  6000.0  DA:  3000.0
Overtime:  1000.0
Incentive:  2000.0
Gross salary:  42000.0
Deduction:  1200.0
Net salary:  40800.0
```

8. Flowchart

START

Input basic salary, overtime hours and incentive

Are any inputs negative?

YES

Display "Inputs cannot be negative"

NO

Calculate HRA, DA and overtime

Calculate gross salary

Calculate deduction and net salary

Display salary statement

STOP

9. Output Screenshot

I can’t generate a screenshot file in this response. To take one:

1. Run the program in Python IDLE or VS Code.
2. Enter the sample input values.
3. After the output appears, press `Win + Shift + S`.
4. Select the output area and save the screenshot.

10. Conclusion

Thus, the Python program calculates the gross salary and net salary of an employee by considering basic salary, HRA, DA, overtime, incentive, and deductions. It displays a detailed salary statement. The program uses input statements, arithmetic operations, conditional statements, and built-in functions.